Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

WebTransport Protocol Fuzzing

Bachelor Thesis

2026
Vladyslav Havriuk

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

WebTransport Protocol Fuzzing

Bachelor Thesis

Study Programme:	Computer Science
Field of Study:	Computer Science
Department:	Department of Computer Science
Supervisor:	doc. RNDr. Martin Stanek, PhD.

Bratislava, 2026

Vladyslav Havriuk

58313450

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Vladyslav Havriuk
Študijný program: informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský

Názov: Protocol fuzzing
Fuzzing protokolov

Anotácia:

Vedúci: doc. RNDr. Martin Stanek, PhD.
Katedra: FMFI.KI - Katedra informatiky
Vedúci katedry: prof. RNDr. Martin Škoviera, PhD.

Dátum zadania: 23.10.2025

Dátum schválenia: 27.10.2025

doc. RNDr. Dana Pardubská, CSc.
garant študijného programu

.....
študent

.....
vedúci práce

Declaration: I hereby declare that I have independently written this entire bachelor thesis entitled "WebTransport Protocol Fuzzing", including all its figures and appendices, using the literature listed in the attached bibliography.

In preparation of the thesis I have also used the following artificial intelligence tools: OpenCode and Anthropic Claude for the following purposes: implementing some of the code segments, and minor editing and formatting of thesis text. I have used artificial intelligence tools in accordance with relevant legal regulations, academic rights and freedoms, ethical and moral principles while simultaneously maintaining academic integrity. I am aware that I am fully responsible for the accuracy of the final text.

Acknowledgments: I would like to thank my supervisor doc. RNDr. Martin Stanek, PhD. for his expert guidance, valuable advice, and comments during the preparation of this thesis.

Abstrakt

Fuzzing protokolov je technika používaná na testovanie implementácií protokolov. Bola úspešne použitá na identifikáciu implementačných chýb a bezpečnostných zraniteľností v rôznych protokoloch. Cieľom tejto práce je preskúmať metódy fuzzingu protokolov a aplikovať ich na reálne implementácie protokolu WebTransport. Výsledky sú analyzované a diskutované.

Táto práca sa zaoberá fuzzingom protokolov na identifikáciu chýb v implementáciách a zraniteľností. Predstavujeme vyvinutý fuzzer založený na BooFuzz pre protokol WebTransport, ktorý zahŕňa mutácie štruktúry správ a duplikáciu sekvencií. Príspevok spočíva v integrácii s aioquic pre QUIC podporu, testovaní na echo serveroch a analýze výsledkov, vrátane objavenia chyby v Rust knižnici. Výsledky ukazujú efektivitu prístupu v odhaľovaní paník v pracovných vláknach, prispievajúc k spoľahlivosti WebTransportu.

Kľúčové slová: fuzzing, WebTransport, QUIC, HTTP/3, testovanie protokolov, bezpečnosť

Abstract

Protocol fuzzing is a technique used to test protocol implementations. It has been successfully employed to identify implementation errors and security vulnerabilities in various protocols. The goal of this thesis is to explore methods of protocol fuzzing and apply them to real-world implementations of the WebTransport protocol. The results are analyzed and discussed.

This thesis addresses protocol fuzzing for identifying bugs in implementations and vulnerabilities. We present a developed fuzzer based on BooFuzz for the WebTransport protocol, which includes message structure mutations and sequence duplication. The contribution lies in the integration with aioquic for QUIC support, testing on echo servers, and analysis of results, including the discovery of a bug in a Rust library. The results demonstrate the effectiveness of the approach in detecting worker thread panics, contributing to the reliability of WebTransport.

Keywords: fuzzing, WebTransport, QUIC, HTTP/3, protocol testing, security

Contents

Introduction **1**

1 Related Work and State of the Art **3**

1.1 Comparison with Our Work 4

2 Protocol Overview, Methodology, and Implementation **6**

2.1 The WebTransport Protocol 6

2.1.1 Layering: QUIC, HTTP/3, and WebTransport 6

2.1.2 Session Establishment 7

2.1.3 QUIC Streams in WebTransport 7

2.1.4 Capsules on the CONNECT Stream 7

2.1.5 Draft Versions and Compatibility 9

2.2 Fuzzing Background 9

2.3 Our Fuzzing Approach 10

2.4 Fuzzing Methodology 10

2.4.1 Oneshot Mode: Single Capsule Mutations 10

2.4.2 Multistep Mode: Scenario-Based Sequence Mutations 12

2.4.3 Scenario-Lastfuzz Mode: Stateful Capsule Fuzzing 14

2.4.4 Version Compatibility of the Fuzzer 14

2.5 Logging and Result Analysis 15

2.5.1 Test Case Database 15

2.5.2 WTFUZZ Structured Server Log Format 15

2.5.3 Offline Log Correlation 16

2.6 Implementation Overview 16

3 Results, Analysis, and Discussion **18**

3.1 Results of the Fuzzing Campaigns 18

3.2 Analysis 19

3.3 Discussion 19

3.3.1 Limitations 19

3.3.2 Comparison with Related Work 19
3.3.3 Suggestions for Improvements 19

Conclusion **21**

References **22**

Appendix A **24**

List of Figures

List of Tables

2.1 QUIC Variable-Length Integer Encoding (RFC 9000, §16) 8
2.2 Multistep scenario step types 12

3.1 Multistep base scenarios 25

Introduction

In this chapter we introduce the topic of protocol fuzzing, motivate the choice of WebTransport as a target, and outline the structure of the thesis.

Protocol fuzzing is a dynamic testing technique used for discovering vulnerabilities in software implementations by providing invalid, unexpected, or random data as inputs to a program. In the context of network protocols, fuzzing helps uncover bugs that static analysis or traditional testing may miss. It is not a replacement for formal verification or exhaustive testing, but rather complements them by simulating realistic attacker conditions, often revealing edge cases such as buffer overflows, crashes, or logic errors arising from malformed messages or unexpected sequences. Fuzzing operates on a feedback-driven basis, where test cases are generated, executed, and mutated based on coverage or crash detection, making it particularly effective for complex systems such as network stacks. However, fuzzing is not infallible; it may miss subtle timing-based issues or require significant computational resources, and its effectiveness depends on the quality of initial seed inputs and mutation strategies.

WebTransport is a modern web API that enables low-latency, bidirectional, and multiplexed communication between web clients and servers, built on top of HTTP/3 and the QUIC transport protocol [7, 12]. It provides features such as streams, datagrams, and sessions for applications like gaming, live streaming, and real-time collaboration. At a high level, WebTransport establishes a session over QUIC, which handles congestion control, multiplexing, and loss recovery, while HTTP/3 provides framing for initial handshakes and capsules. The protocol manages states including connection establishment, stream creation, data transfer, and graceful shutdown, with error handling for invalid frames or capsules. Although WebTransport is still being actively developed according to the latest IETF drafts, it is already widely supported in major browsers such as Chrome, Firefox, and Safari, reflecting its rapid adoption for improving web performance beyond traditional WebSockets. This novelty means that there is not much fuzzing work done in this space, presenting an opportunity to contribute to its reliability at an early stage of the lifecycle.

The chosen black-box/gray-box fuzzing approach is advantageous because it treats the target as a black box without requiring access to source code or instrumentation, making it language-agnostic and suitable for testing diverse implementations such as

those in Python or Rust, while incorporating gray-box elements through knowledge of the protocol specification to guide mutations. This method is well suited for WebTransport because it allows focusing on message structures and sequences without deep integration into specific runtimes, enabling broader applicability across server libraries. We chose to create a universal fuzzer rather than a language-specific one with deep instrumentation because this supports reuse, avoids dependencies on internal states that may differ across implementations, and aligns with the goal of testing echo servers through external logging and crash detection, ensuring that the fuzzer can evolve with the development of the protocol.

The goals of this thesis include developing a fuzzer using BooFuzz [3, 9] with message structure mutation and sequence duplication, testing against echo servers, and analyzing results to identify vulnerabilities, addressing challenges such as handling QUIC encryption and the evolving nature of the protocol.

In Chapter 1 we describe related work and the state of the art in the area of protocol fuzzing. In Chapter 2 we present an overview of the WebTransport protocol, our fuzzing methodology, and the implementation details. In Chapter 3 we present the results, analysis, and discussion.

Chapter 1

Related Work and State of the Art

In this chapter we describe related work in the area of protocol fuzzing and the current state of the art.

The field of protocol fuzzing has evolved significantly, with various techniques aimed at uncovering vulnerabilities in network communications. A comprehensive survey by Chen et al. [13] categorizes protocol fuzzing methods into generation-based, mutation-based, and hybrid approaches, highlighting the role of state-aware fuzzing in handling complex protocol interactions. The authors discuss challenges such as state explosion and the need for efficient seed selection, noting that while traditional black-box fuzzers like Peach and Sulley were foundational, modern tools incorporate AFL-inspired feedback mechanisms to improve coverage. They emphasize that protocol fuzzing differs from general software fuzzing due to the need for modeling message formats and sequences, often requiring domain-specific knowledge to avoid invalid test cases that are immediately rejected. The survey also covers evaluation metrics such as bug discovery rates and coverage depth, and points to limitations in handling encrypted protocols like QUIC, which underlies WebTransport, where decryption hooks or proxying are needed.

A complementary taxonomy is provided by Zheng et al. [14], whose survey of network protocol fuzzers classifies existing tools along dimensions including input generation strategy, protocol knowledge exploitation, and feedback mechanism. The authors systematically compare generation-based and mutation-based approaches, discussing how state awareness and seed scheduling affect bug discovery rates. They identify recurring limitations across the field: most tools assume static protocol specifications and struggle with protocols that evolve across versions—a challenge directly relevant to WebTransport, whose IETF drafts have introduced new capsule types and settings over time. The survey notes that few tools handle multiplexed transports where concurrent streams complicate state tracking, motivating the sequence-aware approach we adopt.

Daniele et al. [4] present LibAFL*, an advanced fuzzing library that supports custom harnesses for protocol testing, demonstrating its use on industrial control systems. They emphasize the importance of non-invasive monitoring, such as log analysis, over invasive instrumentation, which aligns with our language-agnostic approach. The paper compares LibAFL* with BooFuzz, noting BooFuzz's strengths in ease of use for Python-based scripting but weaknesses in handling high-speed protocols without asynchronous integrations. Their findings show that without instrumentation, crash detection relies on external signals such as process termination or log errors, which we adopted for WebTransport servers. A key gap identified is the lack of fuzzers for web-oriented protocols beyond HTTP, with no mention of WebTransport, which underscores the novelty of our contribution.

Finally, Ang and Ranasinghe [1] present QUIC-Fuzz, a greybox fuzzer specifically designed for the QUIC protocol, addressing the challenges that QUIC's encryption, stateful handshake, and strict message validation pose for general-purpose fuzzers. The authors develop a custom harness that integrates with the QUIC stack to generate structurally valid but semantically mutated packets, enabling the fuzzer to reach deeper protocol states. Their work demonstrates that protocol-aware, state-guided fuzzing is significantly more effective than blind mutation for encrypted transport protocols. While QUIC-Fuzz targets the QUIC layer directly, our work operates at the WebTransport layer above it, focusing on capsule semantics and session state rather than transport-level packet structure—the two approaches are thus complementary rather than competing.

1.1 Comparison with Our Work

Compared to these works, our thesis addresses a specific gap in WebTransport fuzzing that none of the surveyed papers directly address, likely due to the protocol's novelty. The surveys by Chen et al. [13] and Zheng et al. [14] provide broad taxonomies of protocol fuzzing but do not include WebTransport examples. QUIC-Fuzz [1] targets the QUIC transport layer but does not extend to WebTransport session semantics above it. Tools like BooFuzz require extensions for QUIC-based protocols, which we implemented using aioquic. Unlike the state-aware approach of LibAFLstar [4], our black/gray-box method avoids language-specific instrumentation, making it more universal but potentially less deep in coverage; it nonetheless proved effective in discovering a reachable assertion bug in the Rust wtransport library [5], where sending a drain capsule without a proper HTTP/3 frame caused a worker thread panic.

Overall, the state of the art reveals a mature ecosystem for protocol fuzzing but with limited application to new web transports, which motivated our development of

a BooFuzz-based fuzzer tailored for WebTransport message structures and sequences.

Chapter 2

Protocol Overview, Methodology, and Implementation

In this chapter we describe the WebTransport protocol in detail, define the fuzzing methodology we employ, and present the design of our experimental framework including test case generation, logging, and result analysis.

2.1 The WebTransport Protocol

WebTransport is a protocol framework that enables web clients to communicate with a remote server using a secure multiplexed transport [7, 12]. It is built on top of HTTP/3 and the QUIC transport protocol, inheriting their properties of encryption, multiplexing, and congestion control. WebTransport provides three data transport mechanisms—bidirectional streams, unidirectional streams, and datagrams—all multiplexed within a single HTTP/3 connection.

2.1.1 Layering: QUIC, HTTP/3, and WebTransport

The layering of WebTransport is central to understanding the protocol. At the lowest level, QUIC [8] provides a secure, multiplexed transport over UDP. QUIC natively supports multiple independent byte streams, each with its own flow control, which eliminates head-of-line blocking between streams. On top of QUIC, HTTP/3 [2] maps HTTP semantics: it defines how QUIC streams carry control data and HTTP request/response messages in the form of frames.

WebTransport uses HTTP/3 only for the initial session handshake. Once a session is established, applications have nearly direct access to QUIC streams. From bottom to top, the stack consists of UDP as the underlying network transport, QUIC providing streams, datagrams, flow control, congestion control, and encryption, HTTP/3 framing

used for the initial CONNECT handshake and capsule exchange on the CONNECT stream, and finally the WebTransport API [12] exposed to web applications.

2.1.2 Session Establishment

A WebTransport session is initiated by a client sending an extended CONNECT request [6] over HTTP/3. The server indicates support for WebTransport by sending a WebTransport-specific setting in its HTTP/3 SETTINGS frame. Both endpoints must also negotiate support for HTTP/3 datagrams and the Capsule Protocol. If the server accepts the CONNECT request (responding with a 2xx status code), the WebTransport session is established. The resulting HTTP/3 stream is called the *CONNECT stream*, and its stream ID serves as the *Session ID* that uniquely identifies the session within the connection.

2.1.3 QUIC Streams in WebTransport

After session establishment, WebTransport data is carried directly on QUIC streams, bypassing HTTP/3 framing for the payload. Each WebTransport stream begins with a small header that links it to the session:

- **Bidirectional streams** begin with a signal value `0x41` (the `WT_STREAM` frame type) followed by the Session ID encoded as a QUIC variable-length integer. The remainder of the stream carries application data.
- **Unidirectional streams** use HTTP/3 unidirectional stream type `0x54`, followed by the Session ID, followed by user-specified data.
- **Datagrams** are sent using HTTP Datagrams, where the payload directly follows the Quarter Stream ID field referencing the CONNECT stream.

The QUIC variable-length integer encoding used for Session IDs and other fields uses 1, 2, 4, or 8 bytes depending on the magnitude of the value, with the two most significant bits of the first byte indicating the encoding length (see Table 2.1). Only the shortest encoding that can represent a given value is considered canonical; longer encodings of the same value (overlong encodings) are technically valid at the wire level but may be handled inconsistently by implementations.

2.1.4 Capsules on the CONNECT Stream

The CONNECT stream that established the session carries *capsules*—typed, length-prefixed messages defined by the Capsule Protocol [10]. Each capsule has the format:

Table 2.1: QUIC Variable-Length Integer Encoding (RFC 9000, §16)

Prefix bits	Byte length	Value range	Example (value 0)
00	1	0–63	0x00
01	2	0–16 383	0x4000
10	4	0–$2^{30} - 1$	0x80000000
11	8	0–$2^{62} - 1$	0xc000000000000000

```
Capsule {
  Capsule Type (i),
  Capsule Length (i),
  Capsule Value (..)
}
```

where both the type and length are encoded as QUIC variable-length integers. WebTransport defines several capsule types for session management and flow control:

- `CLOSE_WEBTRANSPORT_SESSION` (type `0x2843`) — terminates a session with an application error code and an optional error message
- `WT_DRAIN_SESSION` (type `0x78ae`) — signals graceful shutdown of a session
- `WT_MAX_STREAMS` — limits the number of streams within a session (analogous to the QUIC `MAX_STREAMS` frame, but at the session level)
- `WT_STREAMS_BLOCKED` — indicates inability to create a stream due to session-level limits
- `WT_MAX_DATA` — establishes a data limit within a session
- `WT_DATA_BLOCKED` — indicates inability to send data due to session-level limits
- `WT_MAX_STREAM_DATA` and `WT_STREAM_DATA_BLOCKED` — prohibited in WebTransport over HTTP/3 (draft-15, §5.4); their receipt *must* be treated as a session error

A critical property of the Capsule Protocol, specified in RFC 9297, is that **unknown capsule types must be silently ignored**. This ensures forward compatibility: when new capsule types are introduced in later draft versions, servers implementing earlier drafts will safely discard them rather than treating them as errors. A server that crashes on an unrecognized capsule type violates this requirement.

2.1.5 Draft Versions and Compatibility

The WebTransport specification has evolved through multiple IETF drafts. Two drafts are particularly relevant:

- **Draft-03** (July 2022): an early version that uses the `SETTINGS_ENABLE_WEBTRANSPORT` setting (value `0x2b603742`), does not define session-level flow control capsules, and uses the upgrade token `webtransport`. Session termination uses the `CLOSE_WEBTRANSPORT_SESSION` capsule (type `0x2843`).
- **Draft-15** (March 2026): the latest version that uses `SETTINGS_WT_ENABLED`, introduces session-level flow control capsules (`WT_MAX_STREAMS`, `WT_MAX_DATA`, etc.), uses the upgrade token `webtransport-h3`, and prohibits certain capsule types (`WT_MAX_STREAM_DATA`, `WT_STREAM_DATA_BLOCKED`).

The key differences between drafts lie in the capsule types exchanged on the CONNECT stream and the SETTINGS parameters used during connection setup. The underlying QUIC stream mechanics—bidirectional streams prefixed with `0x41`, unidirectional streams with type `0x54`, and HTTP datagrams—remain the same across drafts. Because unknown capsule types are silently ignored per RFC 9297 (see Section 2.1.4), the data transport paths and capsule framing are compatible across all draft versions.

2.2 Fuzzing Background

Fuzzing (or fuzz testing) is a dynamic software testing technique that provides invalid, unexpected, or semi-random data as input to a program in order to discover bugs, crashes, or security vulnerabilities. In the context of network protocols, fuzzing targets the message processing logic of an implementation by sending malformed or unexpected protocol messages and observing the server's behavior.

Fuzzers are typically classified along two axes. By *input generation strategy*, generation-based fuzzers construct inputs from a protocol specification or grammar, mutation-based fuzzers modify existing valid inputs, and hybrid approaches combine both. By *level of program knowledge*, black-box fuzzers treat the target as an opaque system with no access to source code or internal state; white-box fuzzers use instrumentation (e.g., code coverage, address sanitizers) to guide input generation; and gray-box fuzzers use partial knowledge, such as protocol specifications, without requiring source-level instrumentation.

Error detection in protocol fuzzing can be *instrumentation-based* (e.g., AddressSanitizer, code coverage counters) or *observation-based* (e.g., monitoring for process crashes,

non-zero exit codes, log anomalies, or failure to respond to probes). Instrumentation-based detection is more precise but requires access to the target's source code and a per-language toolchain. Observation-based detection is less precise but language-agnostic, making it applicable to any target regardless of implementation language.

2.3 Our Fuzzing Approach

We adopt a black/gray-box approach. The fuzzer treats the server as a black box—no source code access, no instrumentation—but leverages knowledge of the WebTransport protocol specification to construct meaningful test cases. This design is motivated by two considerations:

- **Language-agnostic testing.** WebTransport servers are implemented in diverse languages (Python, Rust, Go, C++). A fuzzer that requires per-language instrumentation (e.g., ASAN for C/C++, Miri for Rust) would need to be adapted for each target. Our approach works with any server that speaks the WebTransport protocol.
- **Specification-driven mutations.** Rather than random byte flipping, we use knowledge of the capsule format (type/length/payload structure), QUIC variable-length integer encoding, and the protocol state machine to generate mutations that are likely to exercise interesting code paths—boundary values, overlong encodings, cross-layer type confusion, and state-violating sequences.

For error detection, we rely on external observation: process crashes, non-zero exit codes, thread panics visible in server logs, and failure to respond to health-check probes. As discussed above, this observation-based approach is less precise than instrumentation but sufficient for detecting robustness defects such as panics, assertion failures, and unhandled error conditions—and it allows us to test servers in any language without modification.

2.4 Fuzzing Methodology

2.4.1 Oneshot Mode: Single Capsule Mutations

The oneshot mode sends a single malformed capsule per connection on the CONNECT stream. Each test case consists of one `[CapsuleType][CapsuleLength][Payload]` blob. The fuzzer independently varies the capsule type and the length-plus-payload fields, drawing values from a shared pool of interesting byte sequences. After sending, a health check (echo probe on a fresh bidirectional stream) verifies that the server is still alive; a failure to respond indicates the server has entered a bad state.

Mutation value pool. The fuzzer constructs a unified pool of byte sequences that is used for *both* the capsule type and the length/payload fields. The pool contains four categories of values:

1. **Valid capsule types** in their canonical VarInt encoding. For example, `CLOSE_SESSION` is encoded as `0x6843` (the canonical 2-byte VarInt for value `0x2843`), and `DRAIN_SESSION` as `0x800078ae` (the canonical 4-byte VarInt for `0x78AE`).

2. **Interesting numeric boundaries** encoded as QUIC VarInts. These exercise boundary conditions in VarInt parsing:

```
Value 63  -> 0x3f           (1-byte max)
Value 64  -> 0x4040         (2-byte min)
Value 16383 -> 0x7fff       (2-byte max)
Value 16384 -> 0x80004000   (4-byte min)
Value 2^62-1 -> 0xffffffffffffffff  (8-byte max)
```

3. **Malformed and overlong VarInt encodings.** These test how implementations handle non-canonical encodings—for example, encoding the `CLOSE_SESSION` type (`0x2843`) using an unnecessarily long 4-byte or 8-byte VarInt instead of the canonical 2-byte form:

```
Canonical:  0x6843             (2 bytes)
Overlong:   0x80002843         (4 bytes, same value)
Overlong:   0xc000000000002843 (8 bytes, same value)
```

   Other malformed values include truncated VarInts (e.g., only 2 of the 4 bytes required by the prefix), the maximum 8-byte value `0xffffffffffffffff` (which exceeds $2^{62} - 1$ and is invalid), and an empty field (zero bytes).

4. **Cross-layer type confusion values.** QUIC frame types (RFC 9000, §19) and HTTP/3 frame types (RFC 9114, §7.2) are injected as capsule type bytes to test that the implementation correctly treats the CONNECT stream as capsule data and does not misinterpret these bytes as lower-layer control frames. Examples include the QUIC `MAX_DATA` frame type (`0x10`), the HTTP/3 `SETTINGS` frame type (`0x04`), and HTTP/3 reserved "grease" values such as `0x21` that must be ignored.

Two-axis test case generation. The fuzzer combines these values along two independent axes. On the first axis, the capsule type varies while the length and payload are held at a neutral default (length = 0, empty body). On the second axis, the capsule type is fixed to a neutral value (`0x00`, an unknown type that must be ignored per RFC 9297) while the length/payload field varies—either as a bare fuzzed length with no body, or as a correctly-lengthed payload drawn from the pool. For example:

```
Axis 1 - fuzz type:     [0x80002843][0x00]
         (overlong CLOSE as type, length=0, no payload)

Axis 2 - fuzz payload:  [0x00][0x04][0x6843 0x00]
         (type=0, length=4, payload=valid CLOSE capsule bytes)
```

This two-axis strategy ensures that both the type parser and the length/payload parser are exercised with all pool values without requiring a full cross-product, keeping the test case count manageable at approximately 12 000 test cases.

2.4.2 Multistep Mode: Scenario-Based Sequence Mutations

The multistep mode executes multi-step scenarios that interleave legitimate data operations with capsule sequences on a single live session. Unlike oneshot mode, which tests capsule parsing in isolation, multistep mode is designed to trigger state-machine bugs: use-after-free, state confusion after session close, crash-on-reorder, and similar defects that only manifest when the server has real streams and data in flight.

Step types. Each scenario is a sequence of typed steps:

Table 2.2: Multistep scenario step types

Step	Action
`bidi(payload)`	Opens a bidirectional WebTransport stream, sends `payload`, awaits echo (1 s timeout, non-fatal)
`uni(payload)`	Opens a unidirectional stream, sends `payload` (fire-and-forget)
`datagram(payload)`	Sends a QUIC datagram
`capsule(data)`	Sends raw capsule bytes on the CONNECT stream
`sleep(duration)`	Pauses between steps

Base scenarios. There are 12 base scenarios, each a manually designed sequence that exercises a specific usage pattern or edge case. For example, the `bidi_then_shutdown` scenario establishes a real echo exchange before tearing down the session:

```
bidi_then_shutdown:
  1. bidi("HELLO")           -- open stream, await echo
  2. capsule(DRAIN_SESSION)  -- signal graceful shutdown
  3. capsule(CLOSE_SESSION)  -- terminate session
```

Other scenarios include `all_transports_then_close` (all three transport paths then teardown), `capsule_between_bidi` (inject a `MAX_DATA` capsule between two echo exchanges), `drain_mid_data` (drain while streams are active), `rapid_bidi_close` (rapid stream creation before close), and `kitchen_sink` (all six action types for maximum permutation surface). The full list of 12 scenarios is in Appendix 3.3.3.

Mutation engine. From each base scenario, the mutator generates all interesting variants using the following strategies:

1. **Permutations**: all orderings of the steps (capped at 120 per scenario to bound combinatorial growth). For the 3-step `bidi_then_shutdown`, this yields 6 permutations, including the reversed order `[CLOSE, DRAIN, bidi]` which tests whether the server handles a close before any data.

2. **Duplications**: each step is duplicated in place and also appended at the end. For example, duplicating the `DRAIN` step:

   ```
   bidi -> DRAIN -> DRAIN -> CLOSE   (in-place)
   bidi -> DRAIN -> CLOSE -> DRAIN   (at end)
   ```

3. **Omissions**: each step is removed in turn. Omitting the `DRAIN` yields `[bidi, CLOSE]`—a close without prior drain.

4. **Rapid-fire**: each non-sleep step is repeated 2, 5, and 10 times. For example, 10 consecutive `bidi("HELLO")` calls test rapid stream creation.

5. **Prohibited capsule injection**: well-formed but prohibited capsules (`WT_MAX_STREAM_DATA` and `WT_STREAM_DATA_BLOCKED`, which per draft-15 §5.4 must cause a session error) are injected at every position in the sequence. For a 3-step scenario, this yields $2 \times 4 = 8$ injection variants.

6. **Post-CLOSE activity**: for scenarios containing a `CLOSE_SESSION` capsule, additional steps are appended after the close—a bidirectional stream, a unidirectional stream, a datagram, a drain capsule, and each prohibited capsule—testing whether the server correctly rejects activity on a closed session.

All generated variants are deduplicated by content. The mutator produces approximately 920 unique test cases across all 12 scenarios.

2.4.3 Scenario-Lastfuzz Mode: Stateful Capsule Fuzzing

The scenario-lastfuzz mode combines the state-richness of multistep scenarios with oneshot-style capsule mutation. Each test case plays a scenario's prefix steps (all steps except the last) as legitimate traffic to put the server into a specific protocol state, then replaces the last step with a single malformed capsule drawn from the same value pool used by oneshot mode.

For example, given the `bidi_then_shutdown` scenario (bidi → drain → close), the prefix `[bidi, drain]` is executed normally, placing the server in a post-drain state. Then a fuzzed capsule such as an overlong-encoded `CLOSE_SESSION` or a truncated VarInt is sent:

```
1. bidi("HELLO")                -- legitimate traffic
2. capsule(DRAIN_SESSION)       -- legitimate drain
3. capsule(c00000000000284300) -- fuzzed: [Type=overlong CLOSE][Len=0]
```

The number of test cases is the product of the 12 scenarios and the number of fuzz values from the oneshot pool. Unlike multistep mode, scenario-lastfuzz does not permute or mutate the step order—it tests capsule-level robustness in each reachable protocol state.

2.4.4 Version Compatibility of the Fuzzer

A key design property of the fuzzer is its universality across draft versions. The tested servers implement different drafts: the Python echo server (based on aioquic) implements draft-03, while the Rust echo server (based on wtransport) implements draft-09. Despite these differences, the same fuzzer targets all servers simultaneously.

This universality rests on three protocol properties described in Section 2.1.5. First, the QUIC stream mechanics are identical across all drafts: bidirectional streams prefixed with `0x41` and Session ID, unidirectional streams with type `0x54` and Session ID, and QUIC datagrams. Second, the capsule format (type/length/payload) is the same across drafts; only the set of defined capsule types changes. Third, and most importantly, per RFC 9297 unknown capsule types must be silently ignored (Section 2.1.4). This last property is fundamental to the fuzzer's design: flow-control capsules introduced in draft-09 and later can be freely sent to a draft-03 server—a compliant server will silently discard them, while a non-compliant server that crashes on an unrecognized capsule reveals a robustness bug.

The fuzzer adapts only the initial connection setup (SETTINGS parameter values and the upgrade token) based on the target's draft version, which is determined before the fuzzing run using a fingerprinting tool.

2.5 Logging and Result Analysis

A systematic approach to logging and result analysis is essential for making sense of thousands of test cases. Our methodology comprises three components: a test case database, a structured server log format, and an offline correlation tool.

2.5.1 Test Case Database

All test cases are recorded to a SQLite database (by default at `boofuzz-results/run_<timestamp>.db`). The database uses the following schema:

```
test_cases
  id              INTEGER PRIMARY KEY
  test_index      INTEGER     -- mutant index
  sent_data       TEXT        -- newline-separated steps
  is_healthcheck  INTEGER     -- 1 for post-test echo probes
  log_group_id    INTEGER FK  -- references log_groups(id)
  timestamp       REAL

log_groups
  id              INTEGER PRIMARY KEY
  fingerprint     TEXT UNIQUE -- SHA-256 of server output
  raw_text        TEXT        -- full server output
```

The `sent_data` field records exactly what was sent to the server, one line per step in the format `action(hex-payload)`:

```
bidi(48454c4c4f)
capsule(800078ae00)
uni(554e49)
datagram(4447)
```

Deduplication via fingerprinting: server output for each test case is hashed with SHA-256. Multiple test cases that produce identical server output share a single `log_groups` row. This grouping is critical for managing the volume of results: instead of examining 12 000+ individual outputs, we analyze a much smaller set of distinct *failure modes*, each represented by a unique fingerprint.

2.5.2 WTFUZZ Structured Server Log Format

To enable correlation between fuzzer-sent test cases and server-side behavior, we define a structured log format. Instrumented servers emit lines to **stdout** in the following format:

```
WTFUZZ|<conn_idx>|EVENT|key1=val1|key2=val2|...
```

where `conn_idx` is a global connection counter (starting at 0) and `EVENT` is one of a defined set of event types. `SERVER_READY` indicates the server is listening (with `bind=host:port`). `SESSION_OPEN` and `SESSION_CLOSE` mark session lifecycle boundaries. `RECV_BIDI`, `RECV_UNI`, and `RECV_DATAGRAM` report data received on the respective transport. `ECHO` records an echo response sent (with type and stream/session ID). `STREAM_RESET` records a stream reset received (with error code).

Standard logging (warnings, stack traces) goes to **stderr** and is not captured by the pipeline. Any server line not starting with `WTFUZZ|` is flagged as `[SERVER RAW]`—these lines typically indicate panics, assertion failures, or unexpected behavior that is of primary interest during analysis.

2.5.3 Offline Log Correlation

After a fuzzing run, the offline correlation tool matches the test case database with the server log:

```
analyze_logs.py --log server.log
                --db boofuzz-results/run_<timestamp>.db
```

The tool parses the server log, matches connection indices to test case indices, and populates the `log_group_id` field in the database. Each test case is associated with the server output it produced, enabling queries such as: "show all test cases that produced a panic" or "group test cases by unique server response." This two-phase approach (fuzzing first, analysis offline) decouples the fuzzer from the analysis logic and allows re-analysis with different criteria without re-running the campaign.

2.6 Implementation Overview

The implementation integrates BooFuzz [3, 9] with aioquic for QUIC support, enabling asynchronous communication over encrypted QUIC connections. BooFuzz provides the session management, test case scheduling, and web-based monitoring interface, while aioquic handles the QUIC handshake, stream creation, and datagram sending.

We extended BooFuzz to support features not present in the original framework: sequence-level mutations (permutation, duplication, omission), custom monitors for health checking via echo probes, and a request logger that writes test cases to the SQLite database described above.

We tested against two echo server implementations: a Python echo server based on aioquic implementing draft-03 [11], and a Rust echo server based on the wtransport library implementing draft-09 [5].

Both servers were instrumented to emit the WTFUZZ structured log format. Crash detection is performed by monitoring process state and server logs, identifying anomalies such as thread panics without a crash of the entire server process.

The source code of the fuzzer is available at `https://github.com/qbitroot/webtransport-fuzzer`.

Chapter 3

Results, Analysis, and Discussion

In this chapter we present the results of the fuzzing campaigns, their analysis, and a discussion of limitations and possible improvements.

3.1 Results of the Fuzzing Campaigns

The fuzzing campaigns revealed several issues, including a reachable assertion bug in the Rust wtransport library [5], where sending a drain capsule without a proper HTTP/3 frame caused a worker thread panic. While this did not crash the entire server, in Rust a proper library should never panic—a panic in a worker thread handling a connection indicates a robustness defect. The analysis showed that sequence mutations were more effective in uncovering state-related bugs than random changes.

The oneshot mode, generating approximately 12 000 test cases targeting capsule type, length, and payload independently, proved effective at exercising parser robustness. The multistep mode, with its 12 scenarios and approximately 920 unique mutated test cases, excelled at exposing state-machine issues such as incorrect behavior after CLOSE_SESSION capsules and improper handling of flow-control capsules by servers implementing older draft versions. The scenario-lastfuzz mode, which combines multistep state setup with oneshot-style capsule mutations, provided additional coverage by testing malformed capsules in specific protocol states that the server reaches only after legitimate traffic.

Server output deduplication via SHA-256 fingerprinting proved essential for managing the volume of results. Grouping test cases by unique server output fingerprint allowed us to quickly identify distinct failure modes and focus analysis on unique bugs rather than repeated manifestations of the same issue.

3.2 Analysis

The analysis of results focuses on evaluating the effectiveness of different mutation strategies. We examine which types of mutations are most effective in uncovering bugs in WebTransport implementations. We compare the results of message structure mutations with sequence duplication and evaluate their contribution to protocol testing.

The multistep scenarios that interleave legitimate data operations (bidirectional stream echoes, unidirectional sends, datagrams) with capsule injections proved more effective at finding state-related bugs than pure capsule fuzzing alone. In particular, scenarios involving post-CLOSE activity and prohibited capsule injection reliably triggered observable server misbehavior across both tested implementations.

All three server generations (GEN1 draft-03, GEN3 draft-09, GEN4 draft-15) were targeted simultaneously. Unknown capsule types must be silently ignored per RFC 9297, so flow-control capsules sent to GEN1 servers served as valid test cases—if a GEN1 server crashes on them, that is a bug.

3.3 Discussion

3.3.1 Limitations

The approach has several limitations. It relies on external logs instead of internal instrumentation, which may miss silent errors that do not surface as crashes or log anomalies. QUIC's encryption layer presents challenges, as intercepting or replaying traffic requires specialized hooks. Finally, without source-level instrumentation, the depth of coverage is inherently limited compared to white-box approaches.

3.3.2 Comparison with Related Work

Compared to protocol-specific tools such as QUIC-Fuzz [1], our approach is more universal but operates at a higher protocol layer. The language-agnostic design enables testing diverse WebTransport implementations without modifying the fuzzer for each target. Unlike white-box approaches that require per-language integration (e.g., ASAN for C/C++, Miri for Rust), our approach works with any server implementation that follows the WebTransport specification and outputs structured logs.

3.3.3 Suggestions for Improvements

Several directions could improve the fuzzer. Extending the campaign to additional WebTransport implementations would broaden the test surface. Client-side fuzzing in browsers would complement the current server-focused approach. When source access

is available, integrating coverage-guided feedback would allow the fuzzer to focus on unexplored code paths. Longer campaigns with larger seed corpora would also increase the probability of finding rare bugs.

Conclusion

In this chapter we summarize the contributions of the thesis and outline directions for future work.

This thesis addressed fuzzing of the WebTransport protocol with the goal of identifying bugs in implementations and vulnerabilities. We presented a developed fuzzer based on BooFuzz that includes message structure mutations and sequence duplication. We developed a universal fuzzer for WebTransport with aioquic integration for QUIC support, applied mutation and sequence duplication techniques tailored to the protocol, and analyzed the results. The analysis revealed bugs in implementations, including a reachable assertion bug in the Rust wtransport library, and demonstrated the effectiveness of the black/gray-box approach in detecting worker thread panics.

Our contribution strengthens protocol testing and contributes to the reliability of WebTransport at an early stage of its lifecycle. The thesis reflects important lessons about the significance of a language-agnostic approach when testing evolving standards.

The broader impacts on network security underscore the need for early fuzzing of new web technologies to prevent vulnerabilities in widely adopted systems. Since WebTransport is already implemented in major browsers but is still being actively developed, timely identification and fixing of bugs is critical for its safe deployment.

Future work could include extending the fuzzer with smarter mutation strategies leveraging machine learning, client-side fuzzing in browsers, and application to additional implementations of the protocol. The source code of the fuzzer is available at `https://github.com/qbitroot/webtransport-fuzzer`.

Bibliography

[1] Kian Kai Ang and Damith C. Ranasinghe. QUIC-Fuzz: An effective greybox fuzzer for the QUIC protocol. *arXiv preprint*, 2025. arXiv:2503.19402. Available at `https://arxiv.org/abs/2503.19402`.

[2] M. Bishop. HTTP/3. RFC 9114, 2022. DOI: 10.17487/RFC9114. Available at `https://www.rfc-editor.org/rfc/rfc9114`.

[3] BooFuzz Contributors. BooFuzz documentation, 2024. Available at `https://boofuzz.readthedocs.io/en/stable/index.html`.

[4] Cristian Daniele, Timme Bethe, Marcello Maugeri, Andrea Continella, and Erik Poll. LibAFLstar: Fast and state-aware protocol fuzzing. In *Proceedings of the 30th European Symposium on Research in Computer Security (ESORICS 2025)*, 2025. Available at `https://conand.me/publications/daniele-libaflstar-2025.pdf`.

[5] Biagio Festa. wtransport GitHub repository, 2024. Available at `https://github.com/BiagioFesta/wtransport`.

[6] R. Hamilton. Bootstrapping WebSockets with HTTP/3. RFC 9220, 2022. DOI: 10.17487/RFC9220. Available at `https://www.rfc-editor.org/rfc/rfc9220`.

[7] IETF. WebTransport over HTTP/3, 2024. IETF Draft. Available at `https://datatracker.ietf.org/doc/draft-ietf-webtrans-http3/`.

[8] J. Iyengar and M. Thomson. QUIC: A UDP-based multiplexed and secure transport. RFC 9000, 2021. DOI: 10.17487/RFC9000. Available at `https://www.rfc-editor.org/rfc/rfc9000`.

[9] Joshua Pereyda. BooFuzz GitHub repository, 2024. Available at `https://github.com/jtpereyda/boofuzz/`.

[10] D. Schinazi and L. Pardue. HTTP datagrams and the capsule protocol. RFC 9297, 2022. DOI: 10.17487/RFC9297. Available at `https://www.rfc-editor.org/rfc/rfc9297`.

[11] W3C. W3C WebTransport echo server example, 2024. Available at https://github.com/w3c/webtransport/blob/main/samples/echo/py-server/ipv4_echo_server.py.

[12] W3C. WebTransport API, 2024. W3C Recommendation. Available at https://www.w3.org/TR/webtransport/.

[13] Xiaohan Zhang, Cen Zhang, Xinghua Li, Zhengjie Du, Bing Mao, Yuekang Li, Yaowen Zheng, Yeting Li, Li Pan, Yang Liu, and Robert H. Deng. A survey of protocol fuzzing. *ACM Computing Surveys*, 57(2), 2024. Available at https://arxiv.org/abs/2401.01568.

[14] Chenyi Zheng, Ye Wang, Hao Huang, Yong Wang, Hao Chen, and Qiang Wei. Survey of network protocol fuzzers: Taxonomy, techniques, and directions. *Computers & Security*, 159:104683, 2025. Available at https://doi.org/10.1016/j.cose.2025.104683.

Appendix A: Contents of the Online Appendix

The online appendix attached to this work contains the source code and data files used in the analysis. The source code is available at `https://github.com/qbitroot/webtransport-fuzzer`.

The repository contains:

- `main.py` — CLI entry point and BooFuzz session setup
- `src/` — fuzzer core: capsule definitions, sequence mutator, connection handler, WebTransport client, health check monitor, request logger, and server manager
- `analyze_logs.py` — offline log correlation tool
- `get-server-version.py` — server draft-version fingerprinting tool
- `server/aioquic/` — Python echo server (draft-03)
- `server/wtransport/` — Rust echo server (draft-09)

To run the fuzzer, use `uv run python main.py` with the appropriate flags as described in the project README.

Multistep Fuzzing Scenarios

Table 3.1 lists all 12 base scenarios used in the multistep and scenario-lastfuzz fuzzing modes. Each scenario is a manually designed sequence of steps that exercises a specific protocol usage pattern or edge case. The mutation engine (Section 2.4.4) generates all interesting variants from each scenario.

Table 3.1: Multistep base scenarios

Scenario	**Steps**	**Target behavior**
`bidi_then_shutdown`	bidi → drain → close	Close race with active echo
`all_transports_then_close`	bidi + uni + dgram → close	All 3 paths, then teardown
`capsule_between_bidi`	bidi → MAX_DATA → bidi	Capsule disrupts stream processing
`drain_mid_data`	bidi → drain → bidi + uni	Drain while streams are active
`flow_with_streams`	MAX_STREAMS → bidi → MAX_DATA → uni	Flow control with live data
`close_error_after_data`	bidi + dgram → close(42, msg)	Error code parsing after activity
`rapid_bidi_close`	bidi × 3 → close	Rapid stream creation before close
`datagram_flood_drain`	dgram × 3 → drain	Drain during datagram processing
`uni_capsule_interleave`	uni/capsule alternating × 5	Capsules don't corrupt uni handling
`kitchen_sink`	all 6 action types	Maximum permutation surface
`contradictory_limits`	MAX_DATA(0) → bidi → MAX_DATA(64K) → bidi	Limit changes with live data
`blocked_then_create`	BLOCKED → bidi → BLOCKED → uni	Streams despite blocked signal